Ex. No: 8

Title: Two-Layer Neural Network with Backpropagation

Packages: NumPy, Matplotlib

AIM:

To implement a two-layer neural network using NumPy and train it with the backpropagation algorithm to solve the XOR problem.

ALGORITHM:

1. Start the program and import NumPy and Matplotlib.
2. Take the XOR inputs X and targets y.
3. Initialize the weights W1, W2 and the biases b1, b2.
4. Forward propagation: h = sigmoid(X.W1 + b1) and output = sigmoid(h.W2 + b2).
5. Calculate the loss (Mean Squared Error).
6. Backpropagation: calculate the error gradients of the output layer and the hidden layer.
7. Update the weights and biases using the gradients.
8. Repeat steps 4 to 7 for 5000 epochs.
9. Print the predictions and plot the loss curve.
10. Display the output and stop.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

# XOR data
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

np.random.seed(1)
W1 = np.random.randn(2, 4)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1, 1))
lr = 1.0

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

losses = []
for i in range(5000):
    # Forward propagation
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    losses.append(np.mean((y - out) ** 2))

    # Backpropagation
    d2 = (out - y) * out * (1 - out)
    d1 = (d2 @ W2.T) * h * (1 - h)

    # Update weights
    W2 -= lr * h.T @ d2
```

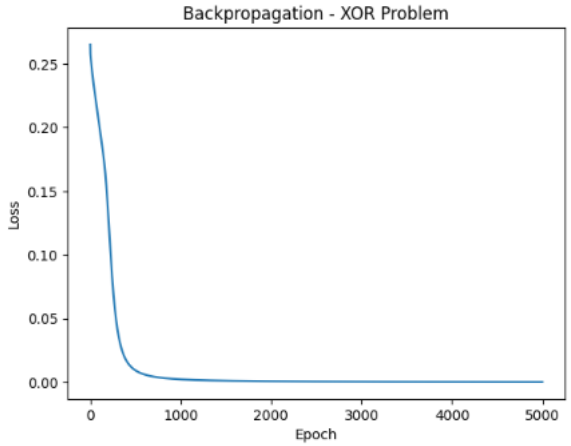
```
    b2 -= lr * d2.sum(axis=0, keepdims=True)
    W1 -= lr * X.T @ d1
    b1 -= lr * d1.sum(axis=0, keepdims=True)

print("Predictions:")
print(np.round(out, 3))
print("Final loss =", round(losses[-1], 5))

plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Backpropagation - XOR Problem")
plt.show()
```

OUTPUT:

```
Predictions:
[[0.008]
 [0.986]
 [0.984]
 [0.021]]
Final loss = 0.00025
```



RESULT:

The two-layer neural network was trained successfully using backpropagation. The loss decreased and the network predicted the XOR output correctly.